ELSEVIER

Simulation Practice and Theory 9 (2001) 21–35

SIMULATION PRACTICE AND THEORY

www.elsevier.com/locate/simpra

A simulation-model compiler for all seasons

Granino A. Korn

ECE Department, University of Arizona, 7750 South Lakeshore Road #15, Chelan, WA 68816, USA

Received 30 January 2001; received in revised form 15 June 2001

Abstract

A new runtime simulation-model compiler quickly reads, compiles, and immediately solves vector differential equations and difference equations as well as scalar equations for dynamic-system models. Vector assignments compactly model nonlinear as well as linear matrix/vector relations for many problems in physics, control systems, neural networks, and fuzzy logic. The key feature is that the new vector differential-equation solver can replicate a dynamic-system model many times and then simulates hundreds of such models in a single simulation run, e.g. for Monte-Carlo studies. Also, partial differential equations (e.g. for a heat exchanger) can be solved together with ordinary differential equations. Portable C code runs on UNIX or LINUX workstations. A fast machine-language version for personal computers (Windows 98 and NT) directly utilizes the Pentium floating-point hardware stack, which is automatically continued into memory. © 2001 Elsevier Science B.V. All rights reserved.

Keywords: Interactive simulation; Vector-operation compiler; Monte-Carlo simulation; Neural-network simulation; Partial differential equations

1. Introduction

This report is organized as follows. Section 2 briefly reviews dynamic-system modeling with ordinary differential equations and the experiment-protocol program that exercises the differential-equation model to try the effects of different parameters and initial conditions. Section 3 continues the review to explain modern windows-type software designed for fast interactive experiments and exhibits a simple differential-equation-solving program.

Sections 3 and 4 then explore new territory. We introduce a novel vector compiler that reads vector and matrix equations, differential equations, and difference equations written much as in standard mathematics. A single simple program line can

E-mail address: gatmkorn@aol.com (G.A. Korn).

0928-4869/01/$ - see front matter © 2001 Elsevier Science B.V. All rights reserved.
PII: S0928-4869(01)00042-8

compile into many hundreds of equations or ordinary differential equations for statistical (Monte-Carlo) studies, neural-network simulation, or partial differential equations. The compiled vector loops are unrolled, so that there is no runtime loop overhead. We exhibit several examples of useful simulations. Section 5 discusses compilation of submodels, again without any runtime overhead. Finally, Section 6 outlines the programming techniques needed to write such a fast (and portable) compiler for personal computers and UNIX workstations.

2. Dynamic-system simulation

Simulation in teaching, research, and design is *experimentation with models*. To study dynamic systems in physics, chemistry, physiology, and aerospace, chemical, and control engineering, one usually models delayed interactions between physical *state variables* x_i with first-order ordinary differential equations (*state equations*)

$$(\mathrm{d}/\mathrm{d}t)x_i = F_i(t; x_1, x_2, \ldots; y_1, y_2, \ldots) \quad (i = 1, 2, \ldots, n), \tag{1a}$$

where t is the system time, and

$$y_j = G_j(t; x_1, x_2, \ldots; y_1, y_2, \ldots) \quad (j = 1, 2, \ldots, m) \tag{1b}$$

are "defined variables" typically representing intermediate results or system outputs. Some or all of the relations (1a), (1b) may be formulated in vector/matrix form,

$$(\mathrm{d}/\mathrm{d}t)x = F(t; x, y), \quad y = G(t; x, y). \tag{1c}$$

Other dynamic-system models (e.g. for digital control systems) employ difference equations, or both difference equations and differential equations.

A *simulation run* solves the system (1a), (1b) to produce *time histories* $x_i(t)$ and $y_i(t)$ in successive time-steps, starting at $t = t_0$ with given *initial values* $x_i(t_0)$. The run ends at $t_0 + \mathrm{TMAX}$. The computer may also calculate functionals, typically performance measures such as a final value $x_i(\mathrm{TMAX})$, or a mean-square error over the simulation run.

An *experiment-protocol* program sets parameters and initial conditions and then "exercises" the *simulation model* (1a), (1b) in successive simulation runs. The experiment protocol can change parameters and/or initial conditions for successive simulation runs. Programmed or command-mode experiment protocols can also use simulation results to program in statistical or optimization studies.

3. Software for interactive modeling

Some popular simulation programs (SIMULINK™, VISSIM™) permit graphical entry of block-diagram simulation models and then *interpret* the program to permit interactive model changes. Such interpreted programs necessarily run relatively slowly, so that more efficient simulation software (AEGIS/ACSL™, DARE) translates screen-entered system equations (1a), (1b) into Fortran, which is then *compiled*

and linked together with appropriate library routines (e.g. for trigonometric functions). Unfortunately, compilation and linking imposes a 4- to 100-s delay every time a program is entered or modified. These delays interrupt the experimenter's train of thought and are annoying, considering that you may want to modify your model hundreds of times in the course of a single modeling session.

Direct Executing SImulation in REal time (DESIRE) simulation programs introduce a serendipitous compromise. We *interpret* only the experiment protocol, which programs only non-time-critical operations such as parameter setting between simulation runs. But differential-equation solving is time-critical in the extreme. Therefore, when the experiment-protocol program calls the first simulation run with the statement drun, an extra-fast run-time compiler *compiles* the right-hand sides of all model equations (1a), (1b) (*DYNAMIC program segment*) directly into memory. A precompiled integration routine then executes immediately to produce solution graphs within less than 0.1 s, an unnoticeable delay.

Properly designed software hides sophisticated program features from elementary users by defaulting to simple and frequently useful cases. So beginners quickly learn to solve simple differential equations with easily readable programs-like

```
–     DAMPED NONLINEAR HARMONIC OSCILLATOR
– – – – – – – – – – – – – – – – – – – – – – – – – – – – – – – – – –
TMAX = 2 | –                      simulation-run length
k = 30 | r = 1 | –                model parameters
x = 1 | –                         initial value of x (xdot = 0)
drun | –                          make a simulation run
– – – – – – – – – – – – – – – – – – – – – – – – – – – – – – – – – –
DYNAMIC
– – – – – – – – – – – – – – – – – – – – – – – – – – – – – – – – – –
d/dt x = xdot | –                 model equations
d/dt xdot = -k * x * x * x - r * xdot
dispt x | –                       display x versus t
```

When you enter this program and then type run a solution graph appears at once. Note that the first 4 program lines specify the *interpreted experiment-protocol program,* which defines the experiment. The remaining lines specify the *compiled DYNAMIC program segment* that defines the simulation model. dispt x requests a runtime graph. The symbol | is a statement separator, and – is a comment delimiter. The DESIRE screen in Fig. 1 shows a more interesting example.

Please note again that experimenters can immediately observe results of programmed or screen-edited model changes. The interpreted experiment-protocol operations are either *programmed* or *executed in command mode*, say for interactive debugging [2]. DESIRE/2000 solves up to 10,000 ordinary differential equations with fixed- and variable-step Runge–Kutta rules, or up to 200 differential equations with Gear and Adams variable-step/variable-order rules, using double-precision floating

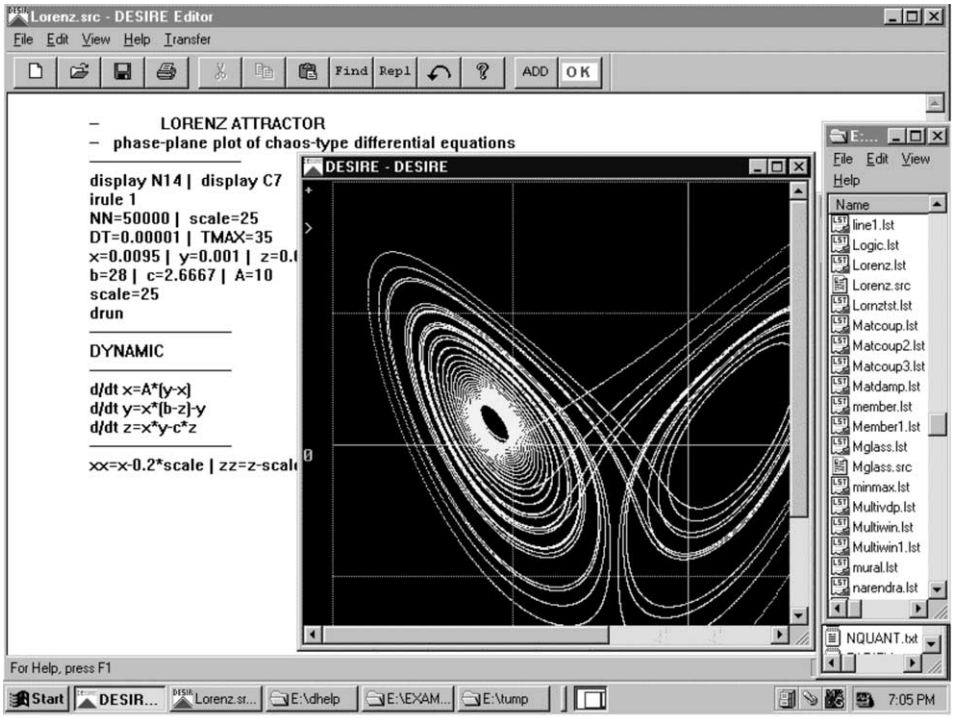

Fig. 1. Solving the Lorenz-attractor differential equations.

point numbers. More advanced users can access more and more features, such as user-defined functions of one or two variables and submodels, iterative multi-run studies, multiple DYNAMIC segments, and vector/matrix differential equations. The main subject of this report, though, is DESIRE's new vector compiler.

4. A new vector compiler

4.1. Vector assignments

Besides scalar linear or nonlinear equations such as

$$\mathsf{y} = \mathsf{A} * \cos(\mathsf{w} * \mathsf{t} + \mathsf{phi}), \quad \mathsf{d/dt\ xdot} = -\mathsf{x} * \mathsf{x} * \mathsf{x} - \mathsf{damping} * \mathsf{xdot}.$$

DESIRE/2000 can declare and manipulate one-dimensional arrays (vectors) and two-dimensional arrays (rectangular matrices). In particular, one-dimensional arrays of subscripted variables $\mathsf{x[k], y[k], \ldots}$ and state variables $\mathsf{q[k]}$ represent vectors $\mathsf{x, y, \ldots q}$, say

$$\mathsf{x} \equiv (\mathsf{x[1], x[2], \ldots, x[n]}).$$

Such arrays are dimensioned in the experiment-protocol program:

$$\mathsf{ARRAY}\ \mathsf{x[n], y[n], z[n], r[m1], s[m2], \ldots} \quad | \quad \mathsf{STATE\ q[n]}.$$

Once declared, vectors can be used both in the interpreted experiment-protocol program and in compiled DYNAMIC segments. In particular, a DYNAMIC-segment model can specify *vector assignments*,

$$\mathsf{Vector\ y = f(x, z, alpha, \ldots)} \quad \text{– compiles into} \quad \mathsf{y[i] = f(x[i], z[i], alpha, \ldots)}$$
$$(\mathsf{i} = 1, 2, \ldots, \mathsf{n}),$$

where $\mathsf{x, y, z, \ldots}$ are all n-dimensional arrays (vectors), and alpha is a scalar. You can have any reasonable number of such Vector assignments with the same or different array dimensions n.

To generate the n vector components $\mathsf{y[i]}$, a compiler loop, which causes no run-time overhead, reads the vector dimension n from the array data structure and compiles the assignment to the ith expression $\mathsf{f(q[i],\ x[i],\ y[I],\ alpha, \ldots)}$ n times, automatically incrementing the vector index i each time.

This example generates a phase-plane plot for the classical Lorenz-attractor set of nonlinear differential equations. The user program was typed into an *Editor Window*. The interpreted experiment-protocol program sets parameters, initial conditions, and integration rule for the compiled differential-equation model in a DYNAMIC program segment.

Existing user programs can be loaded into an Editor Window from its File menu, but it is faster and much more convenient to drag them from the *Explorer Window* on the right. Several user programs, or different versions of the same program, can be open in *multiple Editor Windows*. This makes it easy to compare models. The selected program is activated by a mouse click on its OK button; then another click in the black *Run Window* then compiles and runs the simulation immediately.

The function $\mathsf{f(x[i],\ z[i],\ alpha, \ldots)}$ is the same for all i. It can freely use nonlinear expressions and unlimited nested parentheses. Scalars like alpha may be expressions containing literal numbers as well as symbolic variables. Here is a simple example:

$$\mathsf{Vector\ y = (1 - v) * (cos(alpha * z * t) + 3 * u)} \quad \text{– compiles into}$$
$$\mathsf{y[k] = (1 - v[i]) * (cos(alpha * z[i] * t) + 3 * u[i])} \quad (\mathsf{i} = 1, 2, \ldots, \mathsf{n}).$$

As we shall see, vectors like z can, moreover, be *matrix/vector products* $\mathsf{A * x}$ or *index-shifted vectors* $\mathsf{x\{m\}}$.

4.2. Vector differential equations and difference equations

Given an n-dimensional state vector q and n-dimensional vectors x, y declared as shown above,

$$\mathsf{Mat\ d/dt\ q = f(q, x, y, alpha, \ldots)} \quad \text{– compiles into}$$
$$\mathsf{d/dt\ q[i] = f(q[i], x[i], y[I], alpha, \ldots)} \quad (\mathsf{i} = 1, 2, \ldots, \mathsf{n}).$$

Our vector compiler again loops to generate n derivative components $\mathsf{f(q[i],\ x[i],\ y[I],\ alpha,\ \ldots)}$, which are then inserted into an integration routine.

The n ordinary differential equations can be nonlinear and are integrated automatically, with a choice of integration rules. The experiment protocol can assign initial values for the state variables q[i] or accept the default value 0.

A third vector operation,

```
Delta x = f(x, y, alpha,...)   – compiles into
    x[i] = x[I] + f(x[i], y[i], alpha,...)   (i = 1,2,...,n)
```

compactly implements a vector difference equation useful e.g. for recursive summation.

4.3. High computing speed for parameter studies and Monte-Carlo simulation

Note, at this point, that these vector operations cause *no runtime loop overhead*. They simply compile into multiple scalar assignments (loop-unrolling compiler). DESIRE/2000 currently handles up to 10,000 first-order differential equations.

With a suitable parameter array filled by the experiment-protocol program, we can exercise replicated a models with different parameter values in a single simulation run and tabulate or cross-plot results. But a more interesting application of the Vector and Mat d/dt operations is *Monte-Carlo simulation by model replication* [1]. As a simple example, we can model n = 1000 cannonball trajectories with air resistance [2] and random wind forces:

```
Vector v = sqrt(xdot * xdot + ydot * ydot)   | – computes n airspeeds v[i]
–
Mat d/dt x= xdot | Mat d/dt y =ydot          | – 4n state equations for n models
Mat d/dt xdot = –R * v * xdot + ran()
Mat d/dt ydot= –R * v * ydot –g
```

A single simulation run now generates all 1000 random-input trajectories and readily computes runtime statistics [1]. Since there is no loop overhead, a Pentium-class personal computer produces the equivalent of more than 1000 repeated trajectories in one simulation run (Fig. 2). With an 800-MHz Pentium III this takes about 0.3 s, compared to over 1 s for 1000 repeated Monte-Carlo runs [1]. This permits very fast interactive studies of the effects of parameter changes *on population statistics*, such as mean-square guidance-system errors.

5. Vector/matrix products and index-shifted vectors implement interconnections between model variables

5.1. Going beyond simple model replication

With simple model replication, each vector equation or differential equation, say

```
Mat d/dt q = f(q,x,y,alpha,...)
```

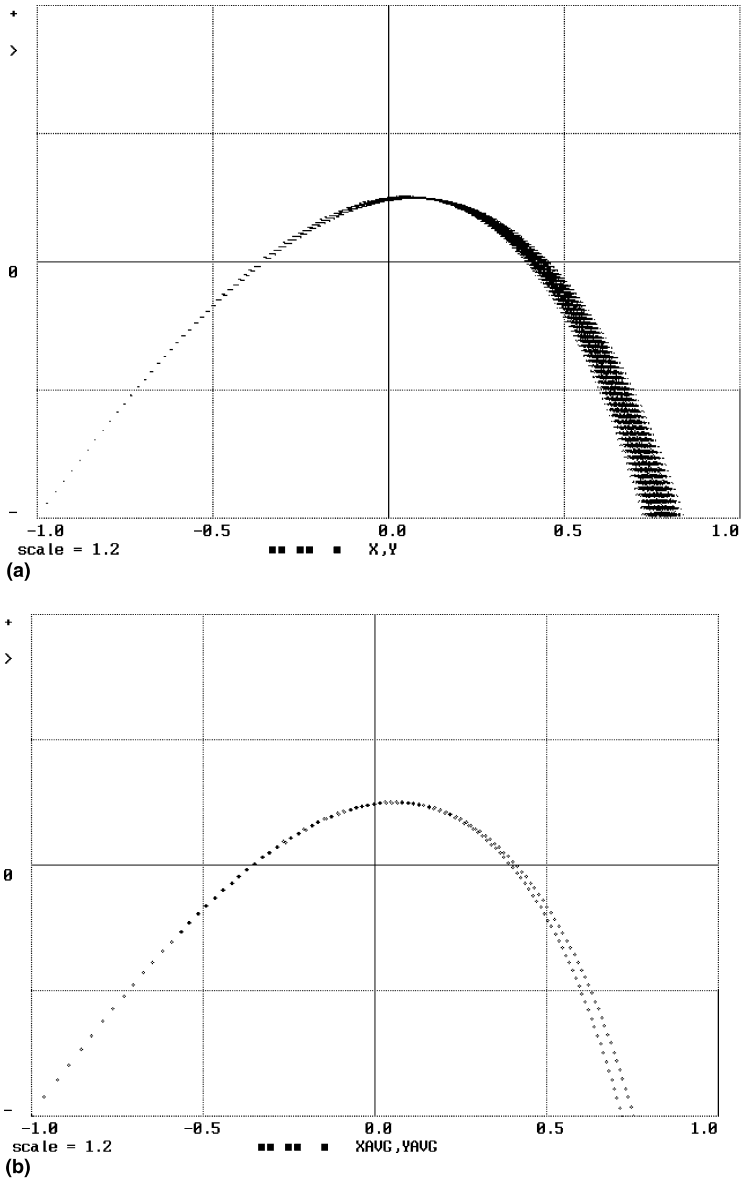

Fig. 2. (a) 500 cannonball trajectories with noisy wind forces were run in succession for this repeated-run Monte-Carlo simulation run. On a 400-MHz Pentium II, this took 1.95 s with the display turned off. (b) Replication of 500 trajectory runs lets you compute Monte-Carlo population averages in a single simulation run (see text), so that the effects of model-parameter changes *on statistics* can be studied. The figure shows average trajectories for two samples of $n = 500$ replicates of the cannonball model. Each simulation run (the equivalent of 500 repeated runs) took 0.25 s on he 400-MHz Pentium II. Note that the sample averages xavg and yavg are themselves random variables.

relates only vector components elements q[i], q[i], q[i], ... all labeled with the same index i. But more sophisticated and useful models must also be able to relate different components of the same vector, and also components of different vectors, which need not have equal dimensions.

5.2. Matrix/vector products relate different vectors

Any vector, say z, on the right-hand side of a Vector, Mat d/dt, or Delta operation can be a *matrix-vector product* A ∗ v, where v can be defined by another Vector assignment. [1] Here A is a rectangular matrix declared with

```
ARRAY A[n, m]
```

and "filled" with array elements A[i,k] assigned by the program.

Simple vector assignments like Vector y = A ∗ x conveniently implement geometrical coordinate transformations, such as rotations. A more interesting example is the vector/matrix representation of a simple nonlinear *two-layer neural network*:

```
ARRAY layer1[n], layer2[m], layer3[r], bias1[n], bias2[m], W2[m, n], W3[r, m],
  ..................
  Vector layer2 = tanh(W2 * layer1 + bias1),
  Vector layer3 = tanh(W3 * layer2 + bias2).
```

This automatically compiles into multiple equations,

$$\mathsf{layer2[i]} = \mathsf{tanh}\left(\sum_{j=1}^{n} \mathsf{W2[i,j]\ layer1[j] + bias1[i]}\right) \quad (i = 1, 2, \ldots, m),$$

$$\mathsf{layer3[k]} = \mathsf{tanh}\left(\sum_{j=1}^{m} \mathsf{W3[k,j]\ layer2[k] + bias2[k]}\right) \quad (k = 1, 2, \ldots, r).$$

Vector and Mat d/dt operations can model classical perceptrons and backpropagation networks, but also sophisticated pulse-coupled (oscillating) neurons [4], as well as recurrent neural networks incorporating feedback and time delays [3]. For this purpose, DESIRE can also compile matrix equations and difference equations used, for instance, to update neural-network connection-weight matrices [3]. Special operations implement competitive neuron layers [3]. Fig. 3 shows two examples.

The same easily readable vector/matrix notation also produces *compact models of linear or nonlinear control systems* with multiple inputs and/or outputs. A good example is the familiar classical state-transition model

[1] If v is identical with assignment target of a Vector or Delta operation, the program reminds you that an intermediate vector assignment Vector z = A ∗ v is needed, just as it would be in Fortran or C. There is no such problem in Mad d/dt operations, since the differencial-equation solver automatically assigns a hidden intermediate variable for each derivative.

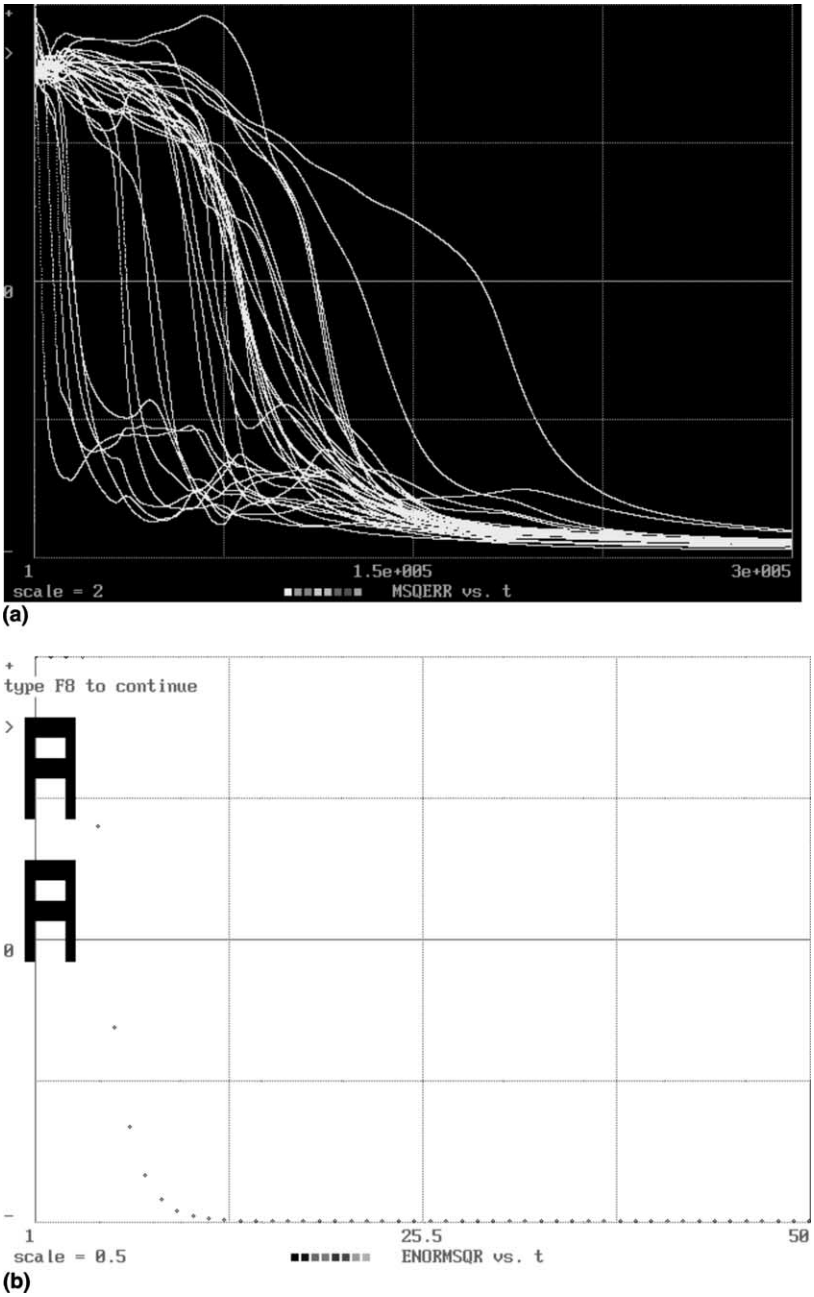

Fig. 3. (a) A backpropagation network learning 32 patterns simultaneously. All 32 squared pattern errors are plotted against time. (b) A character-recognition network learning the pattern for the letter A (based on [3]). The square of the pattern error is plotted against time to show how the error converges.

$$\mathsf{Mat\ d/dt\ q} = \mathsf{A} * \mathsf{u} + \mathsf{B} * \mathsf{u},$$
$$\mathsf{Vector\ r} = \mathsf{C} * \mathsf{q} + \mathsf{D} * \mathsf{u},$$

which relates outputs r[i] and inputs u[k] of a linear control system.

5.3. Index-shift operations

Other important applications relate only a few different components of the same or different vectors, say $\mathsf{x[i]},\ \mathsf{x[i+1]},\ \mathsf{x[i-2]},\ \mathsf{y[i-2]}, \ldots$ We permit any vector on the right-hand side of a Vector, Mat d/dt, and Delta operation to be an *index-shifted vector* denoted by u/m, with components $\mathsf{u[1+m]},\ \mathsf{u[2+m]}, \ldots$ Specifically,

Vector $\mathsf{y} = \mathsf{sin(x\{m\})}$ compiles into [2] $\mathsf{y[i]} = \mathsf{sin(x[i+m])}$ $(\mathsf{i} = 1, 2, \ldots, \mathsf{n})$

The index shift m is an integer expression set at compile time. When $\mathsf{i+m} < 1$ is $>\mathsf{n}$ then $\mathsf{x[i+m]}$ is simply replaced by 0.

As an example, we can declare an n-dimensional vector x with ARRAY x[n] and program

$$\mathsf{output} = \mathsf{x[n]} \quad | \quad \mathsf{Vector\ x} = \mathsf{x\{-1\}} \quad | \quad \mathsf{x[1]} = \mathsf{input}$$

to shift a given scalar input input(t) through $\mathsf{x[n], x[n-1], \ldots, x[1]}$ and then into the scalar output output(t) in successive simulation time-steps. This models a delay line useful, for instance, to shift a time function into transverse filter or predictor network (serial-to-parallel conversion).

DESIRE's index-shifting feature was originally designed to model recurrent neural networks incorporating time delays, lateral inhibition, or adaptive predictors [3]. More recently, index-shifting has also neatly implemented space-coordinate differentiation for *interactive Method-of-lines solution of partial differential equations* (Fig. 4) [5,6]. This lets you directly include models of heat exchangers, simple chemical reactors, etc. in process-control simulations modeled with ordinary differential equations.

6. Desire submodels compile without runtime overhead

Modeling languages are further extended with *user-defined functions* and *submodels*. Submodels implement special function blocks or complete sets of equations and differential equations.

For a Method-of-lines solution of the partial differential equation (heat conduction in a rod, temperature u is a function of x and t)

[2] If v is identical with assignment target of a Vector or Delta operation and $\mathsf{m} > 0$, the program reminds you that an intermediate vector assignment Vector $\mathsf{z} = \mathsf{v\{m\}}$ is needed, just as it would be in Fortran or C. Again, there is no such problem in Mat d/dt operations, since the differential-equation solver auotomatically assigns a hidden intermedite variable for each derivative.

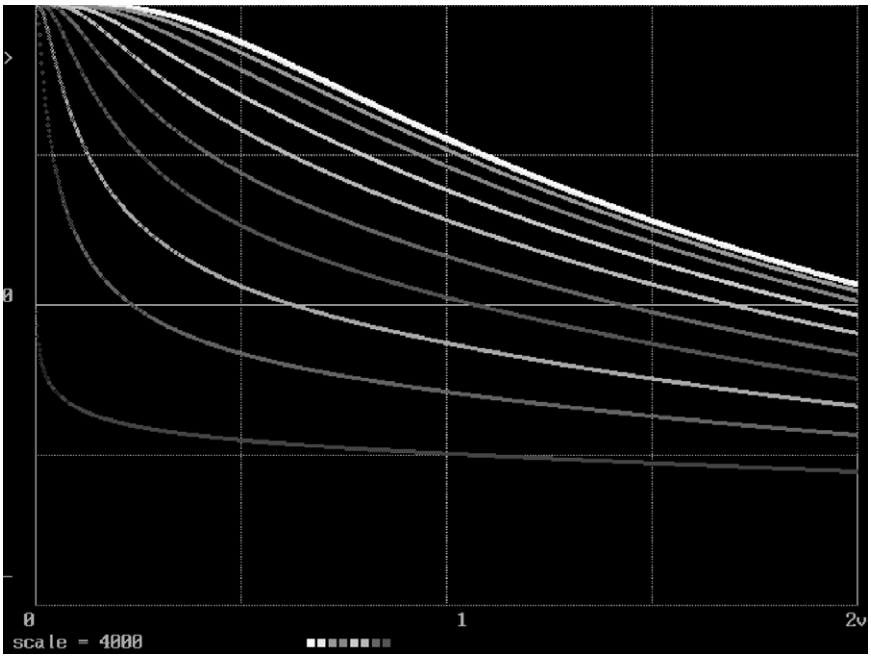

Fig. 4. Solving a partial differential equation by the Method of Lines [6].

$$\partial u/\partial t = uxx \quad (ux \equiv \partial u/\partial x,\ uxx \equiv \partial ux/\partial x)$$

we declare N-dimensional arrays of corresponding node values of x, u, ux, and uxx:

```
ARRAY x[N], u[N], ux[N], uxx[N].
```

A numerical-differentiation submodel uses vector-shift operations to produce the array ux from u and uxx from ux, with modified difference formulas at the boundaries [5]:

```
MACRO DDx(n, b, v, vx)
  –
  Vector vx = b * (v{1} – v{–1})   | –        numerical differentiation
  –
  vx[1] = b * (–3 * v[1] + 4 * v[2] – v[3])   | –   differentiation at boundaries
  vx[n] = b * (3 * v[n] – 4 * v[n – 1] + v[n – 2])
  end
```

The experiment-protocol program sets parameters and initial conditions and then calls a DYNAMIC program segment to solve ordinary differential equations. The boundary condition for x[N] is a nonlinear heat-radiation condition.

```
DYNAMIC
  - - - - - - - - - - - - - - - - - -
  invoke DDx(N, B, u, ux) | –              differentiate u to get ux
  ux[1] = 0 | ux[N] = E * (UA4 - u[N]^4) | –  set boundary values for ux
  invoke DDx(N, B, ux, uxx) | –            differentiate ux to get uxx
  Mat d/dt u = uxx | –                     N ordinary differential equations
```

The program displays solution values u[i] for differentx[i] as functions of the time t representing, say, a small servomechanism or a neural network for convenient reuse. Submodels are much more than a convenient shorthand notation: they are abstractions that make it easier to understand models. A DESIRE *submodel declaration*

```
MACRO F(x$, y$, ...)
  prototype code lines
  ...............
  end
```

includes DYNAMIC-segment lines referencing dummy variables x$,y$,..., which can be scalars, vectors, or rectangular matrices. When a DYNAMIC program segments *invokes* the submodel with

```
INVOKE F(x, y, ...)
```

the invocation reproduces the submodel code with new variables x, y, ... corresponding to the dummy variablesx$, y$,... (Fig. 4). Interestingly, DESIRE submodels are *not* implemented as text-reproducing macros, and they do *not* invoke runtime subroutines. They generate *compiler subroutines* that produce runtime code *without any overhead for function calls*.

7. How it is done: compiler implementation for personal computers and UNIX

7.1. In-line and threaded-code implementations

The earliest DESIRE runtime compilers produced true in-line machine code for the DEC PDP-11and VAX, and then for Intel personal computers. The compiler front end was written in Pascal, but the code generator employed machine-specific assembly language to produce the highest possible execution speed. For the same reason, we used single-precision code to compute derivatives, with double-precision integral accumulation. A machine-language runtime library allowed simulations to outrun personal-computer Fortran.

This was very efficient, but would be prohibitively expensive on RISC-type UNIX workstations, especially since each CPU type would require a new machine-language code generator. For this reason, more recent DESIRE compilers are written in ANSI-C and generate threaded code (virtual-machine code) rather than in-line code. This also makes large vector programs much more manageable. The new compilers

are *portable*: they can be recompiled, with minimal changes, for different computers and operating systems. The system is currently supported with the Windows interface shown in Fig. 1 (Windows 98, Me, NT, and 2000), and as a console program displaying color graphics under SOLARIS™, LINUX, ans MS-DOS. Experimental versions have also run under AIX™, VMS/DECwindows™, and OS/2. We employ double-precision (64-bit) floating-point code throughout, not just for integral accumulation.

Our compilers produce efficient modified stack-machine code, i.e. they generate "add memory to stack top", "multiply memory into stack top" as well as simple push and pop operations. Thus the expression assignment

```
x = a * b + c * d
```

produces optimal "stack-machine code",

```
push a
multiply stack top by b
push c
multiply stack top by d
sadd top stack registers
pop x
```

7.2. Portable C implementation

The portable threaded-code compiler compiles an array

```
void (*COMPILE[N])()
```

of successive function pointers pointing to pseudoinstructions implemented as C functions COMPILE[ci](), and also an array of successive integer data pointers dptr[M] for these functions. In effect, the corresponding array pointers ci and di function as the program counter and address register of a *virtual machine* whose instructions are function calls.

In this way, push a and pop a compile into function pointers which point to

```
data[--ss] = data[dptr[di++]];
data[dptr[di++]] = data[ss++];
```

Similarly, multiply stack top by b points to

```
data[ss] *= data[dptr[di++]];
```

The array data contains double-precision floating point data. The C variable ss serves as the stack pointer of the virtual machine. Functions like sin(x) are returned on the virtual-machine stack.

7.3. Machine-language hardware-stack-machine code generator for personal computers

Our personal computer (Windows) program is largely identical with the UNIX program, except that now the virtual machine is implemented in Pentium-optimized *machine code. The Pentium floating-point processor serves directly as a true stack machine, and the virtual program counter and data address register are real hardware registers*. What is more, assembly language lets us to replace function calls and returns with much more efficient jumps via the Pentium register **ECI**, which conveniently serves as a hardware program counter and is incremented after each jump. We show an example in Appendix A.

The Intel processors used in personal computers have a double-precision floating-point hardware stack with appropriate instructions. But this stack has only 8 directly accessible registers, of which two must be reserved to generate functions such as exponentials. At this point, DESIRE uses a special trick. The compiler maintains a stack counter and compiles the normal processor-stack instructions listed above as long as the stack depth does not exceed 6 registers. But when a long expression requires more than 6 stack registers, we automatically compile modified push and pop routines that continue the processor stack into memory with machine-language push register and pop into register instructions. Since stack counting is done by the compiler it imposes no runtime overhead.

7.4. Execution speed

Vector-compiled Monte-Carlo simulation was 2–3 times faster than the repeated simulation runs needed without vector compilation [2]. This is true because of the saved vector-loop overhead. Since the loop-unrolled code is necessarily larger, the exact speed advantage depends on the amount of L2 cache available to the program.

Threaded-code execution incurs the overhead of indirect data references and repeated function calls. Our special machine-language program reduces the function-call overhead and generates true hardware stack-machine code. In general, threaded machine-language code was found to be only 2–3 times slower than true in-line machine code. Using portable C code instead of machine language slows you down by another factor of 2–3.

With a 400-MHz Pentium II, a simple three-dimensional flight simulation required

0.07 s with true in-line compilation of single-precision machine code,

0.26 s with threaded-code compilation of double-precision machine code,

0.74 s with threaded-code compilation of double-precision C code.

Newer personal computers (Pentium III or Pentium 4 processors with Rambus memory, or ATHLON processors) and high-end UNIX workstations are 3–4 times faster. It seems fair to use some of the processor speed to pay for code portability and maintainability.

Appendix A. Fast threaded-code implementation on personal computers

In Pentium (or Athlon) assembly language, push a now produces

```
MOV EAX, DWORD PTR_dptr[EDI * 4]      ;load 4-byte data address
INC ESI                               ;increment program counter
FLD QWORD PTR_data[EAX * 8]           ;push a (8 bytes)
INC EDI                               ;increment data pointer
JMP DWORD PTR_COMPILE(ESI * 4)        ;next pseudoinstruction
```

and multiply stack top by b points to

```
MOV EAX, DWORD PTR _dptr[EDI * 4]     ;load 4-byte data address
INC ESI                               ;increment program counter
FMUL QWORD PTR_data[EAX * 8]          ;multiply stack top by a (8 bytes)
INC EDI                               ;increment data pointer
JMP DWORD PTR_COMPILE(ESI * 4)        ;next pseudoinstruction
```

The hardware register EDI is the virtual address register. The vital push and pop operations execute on the actual hardware coprocessor stack. Note that we used scaled indexing. INC EDI and INC ESI were transposed to optimize Pentium pipeline operation [7].

References

[1] G.A. Korn, Fast Interactive Monte Carlo Simulation of Dynamic Systems By Model Replication, 1999 (submitted).
[2] G.A. Korn, Dynamic-system Simulation with Microsoft Windows 95 and NT, Gordon and Breach, London, 1998.
[3] G.A. Korn, Neural Networks and Fuzzy-logic Control on Personal Computers and Workstations, MIT Press, Cambridge, MA, 1995.
[4] J.L. Johnson, Mary Lou Padgett, Pulse-coded neural network (PCNN) models and applications, Trans. IEEE Neural Networks 10 (1999) 480–498.
[5] W.E. Schiesser, The Numerical Method of Lines, Academic Press, New York, 1991.
[6] G.A. Korn, Interactive solution of partial differential equations by the method of lines, Math. Comput. Simulation 1644 (1999) 1–10.
[7] Pentium III Programmer's Manual, Intel Corporation, Santa Clara, CA, 1998.